

LLM Notes

2026 Spring

Day 6: 残差连接

deeplearning

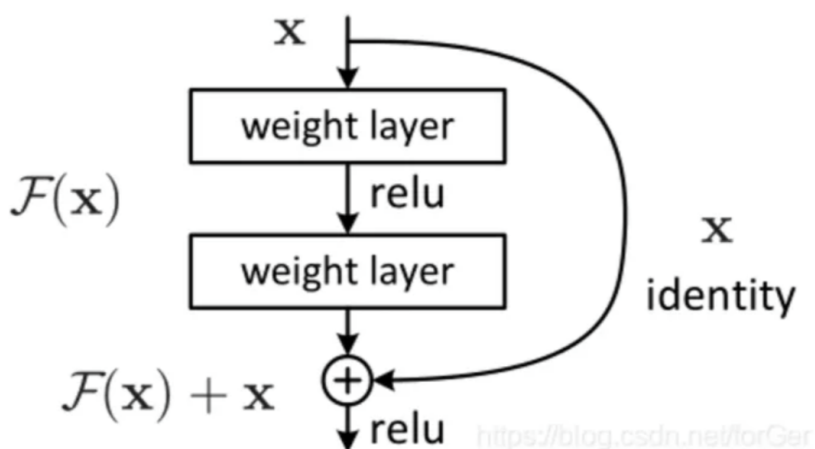
主讲: Aaron

Fudan University

今日摘要

今天介绍残差连接的基本思想

6.1 残差连接架构



6.1.1 残差连接图解析

上图为经典的残差连接图解， x 除了流经正常的线形层、激活函数外，还有一条 skip connection 直连，两者求和后再经过一次 relu 激活。

定义

定义 6.1.1 (残差块). 设输入为 x ，期望输出为 $\mathcal{H}(x)$ ，残差块拟合的映射为：

$$\mathcal{F}(x) = \mathcal{H}(x) - x$$

则最终的输出为：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$$

其中 $\mathcal{F}(\mathbf{x})$ 为残差函数， $\mathbf{x}$ 通过**快捷连接（Skip Connection）**直接与输出相加。

分析

为什么能缓解梯度消失？

反向传播时，损失对输入的梯度包含一项：

$$\frac{\partial E}{\partial \mathbf{x}} = \frac{\partial E}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial E}{\partial \mathbf{y}} \cdot \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + 1 \right)$$

由于 +1 的存在，梯度可以直接“跳过”残差块传播到浅层，即使 $\partial \mathcal{F} / \partial \mathbf{x}$ 很小，梯度也不会完全消失。

6.1.2 BasicBlock 代码分析

```

1 class BasicBlock(nn.Module):
2     expansion = 1 # 输出通道数 = planes * expansion
3
4     def __init__(self, inplanes, planes, stride=1, downsample=
5         None):
6         super(BasicBlock, self).__init__()
7         # 第一个卷积：可能改变空间尺寸 (stride>1)
8         self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=3,
9                                 stride=stride, padding=1, bias=
10                                    False)
11         self.bn1 = nn.BatchNorm2d(planes)
12         self.relu = nn.ReLU(inplace=True)
13         # 第二个卷积：stride=1, 不改变空间尺寸
14         self.conv2 = nn.Conv2d(planes, planes, kernel_size=3,
15                                 stride=1, padding=1, bias=False)
16         self.bn2 = nn.BatchNorm2d(planes)
17         self.downsample = downsample # 用于匹配形状
18         self.stride = stride
19
20     def forward(self, x):
21         identity = x # 保存输入

```

```
21     out = self.conv1(x)
22     out = self.bn1(out)
23     out = self.relu(out)
24
25     out = self.conv2(out)
26     out = self.bn2(out)
27
28     if self.downsample is not None:
29         identity = self.downsample(x) # 调整输入形状, 使其与
30                                     # out 匹配
31
32     out += identity # 残差连接
33     out = self.relu(out)
34     return out
```

Listing 1: BasicBlock: 基础残差块 (ResNet-18/34)

备注

`expansion = 1` 表示 BasicBlock 的输出通道数就是 `planes`, 与输入 `inplanes` 不一定相等, 通过 `downsample` 中的 1×1 卷积调整。

6.1.3 Bottleneck 代码分析

```
1 class Bottleneck(nn.Module):
2     expansion = 4 # 输出通道数 = planes * 4
3
4     def __init__(self, inplanes, planes, stride=1, downsample=
5         None):
6         super(Bottleneck, self).__init__()
7         # 1x1 降维: 减少计算量
8         self.conv1 = nn.Conv2d(inplanes, planes, kernel_size=1,
9                                 bias=False)
10        self.bn1 = nn.BatchNorm2d(planes)
11
12        # 3x3 特征提取: stride 可能 >1
13        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3,
14                                stride=stride, padding=1, bias=
15                                    False)
16        self.bn2 = nn.BatchNorm2d(planes)
```

```

15     # 1x1 升维：恢复（并扩充）通道数
16     self.conv3 = nn.Conv2d(planes, planes * self.expansion,
17                             kernel_size=1, bias=False)
18     self.bn3 = nn.BatchNorm2d(planes * self.expansion)
19
20     self.downsample = downsample
21     self.stride = stride
22
23     def forward(self, x):
24         identity = x
25
26         out = self.conv1(x)
27         out = self.bn1(out)
28         out = self.relu(out)
29
30         out = self.conv2(out)
31         out = self.bn2(out)
32         out = self.relu(out)
33
34         out = self.conv3(out)
35         out = self.bn3(out)
36
37         if self.downsample is not None:
38             identity = self.downsample(x)
39
40         out += identity
41         out = self.relu(out)
42         return out

```

Listing 2: Bottleneck: 瓶颈结构 (ResNet-50/101/152)

分析

BasicBlock vs Bottleneck 对比：

BasicBlock	Bottleneck
两层卷积： $3 \times 3 \rightarrow 3 \times 3$	三层卷积： $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
$\text{expansion} = 1$	$\text{expansion} = 4$
计算量适中, 用于较浅网络 (18/34 层)	先降维后升维，减少计算量，用于深层网络 (50+ 层)

Bottleneck 的设计哲学：先用 1×1 卷积将通道数压缩为原来的 $1/4$ ，在低维空间进行 3×3 卷积，再用 1×1 卷积恢复维度。这大幅减少了 3×3 卷积的计算量，使得网络可以做到 100+ 层。

6.1.4 形状匹配：downsample 的作用

分析

残差连接要求 $\mathcal{F}(\mathbf{x})$ 与 $\mathbf{x}$ 的形状完全相同，但实际中往往不一致：

1. 通道数不一致： $\text{inplanes} \neq \text{planes} * \text{expansion}$
2. 空间尺寸不一致： $\text{stride} > 1$ 时输出高宽减半

解决方案：对输入 $\mathbf{x}$ 进行 1×1 卷积（步幅为 stride ），使其形状与 $\mathcal{F}(\mathbf{x})$ 对齐：

$$\text{identity} = \text{downsample}(\mathbf{x})$$

其中， $\text{downsample} = \text{Conv2d}(\text{inplanes}, \text{planes} \times \text{expansion}, \text{kernel_size} = 1, \text{stride} = \text{stride})$

值得注意：调整的是输入 $\mathbf{x}$ （快捷连接分支），而非残差 $\mathcal{F}(\mathbf{x})$ 分支。